



Legate: A Programming Framework For Scalable and Composable Software

Wonchan Lee | PAW-ATM 2024 | Nov 17, 2024

Vision: Scaling with Zero Code Change

Author in familiar APIs, scale to any machines



Human productivity is the biggest bottleneck for parallel/distributed programming

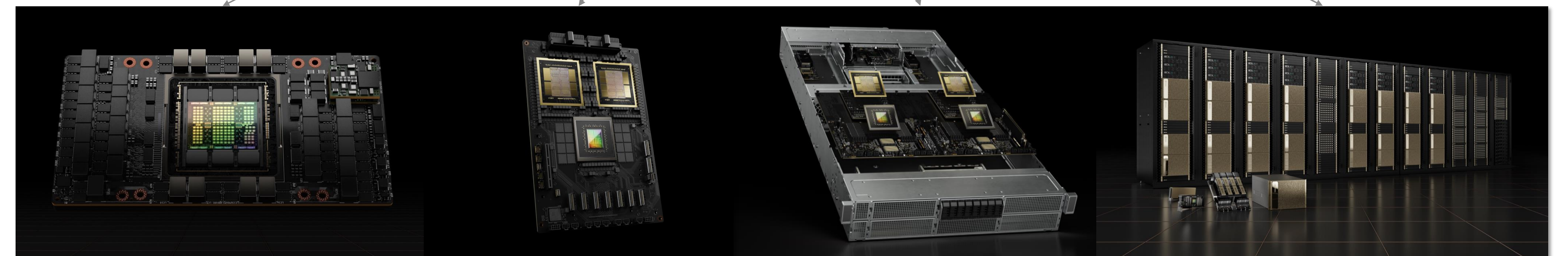
- Prototypes written in “easy” programming languages/APIs (e.g., Python, MATLAB, etc.) are often re-implemented using “more serious” frameworks for scaling

“Zero code change” scaling for better productivity

- Author programs in familiar APIs, scale them to any machines
- Easy transition from prototyping to production

```
# Generate a random positive semi-definite matrix
A = scipy.sparse.random(n, n, format="csr")
A = 0.5 * (A + A.T) + n * scipy.sparse.eye(n)

# Estimate the maximum eigenvalue of A
x = numpy.random.rand(A.shape[0])
for _ in range(iters):
    x = A @ x
    x /= numpy.linalg.norm(x)
result = numpy.dot(x.T, A @ x)
```



Single GPU

Mixed
CPU/GPU

Single-Node
Multi-GPU

Multi-Node Multi-GPU
Cloud & Supercomputer

Introducing Legate

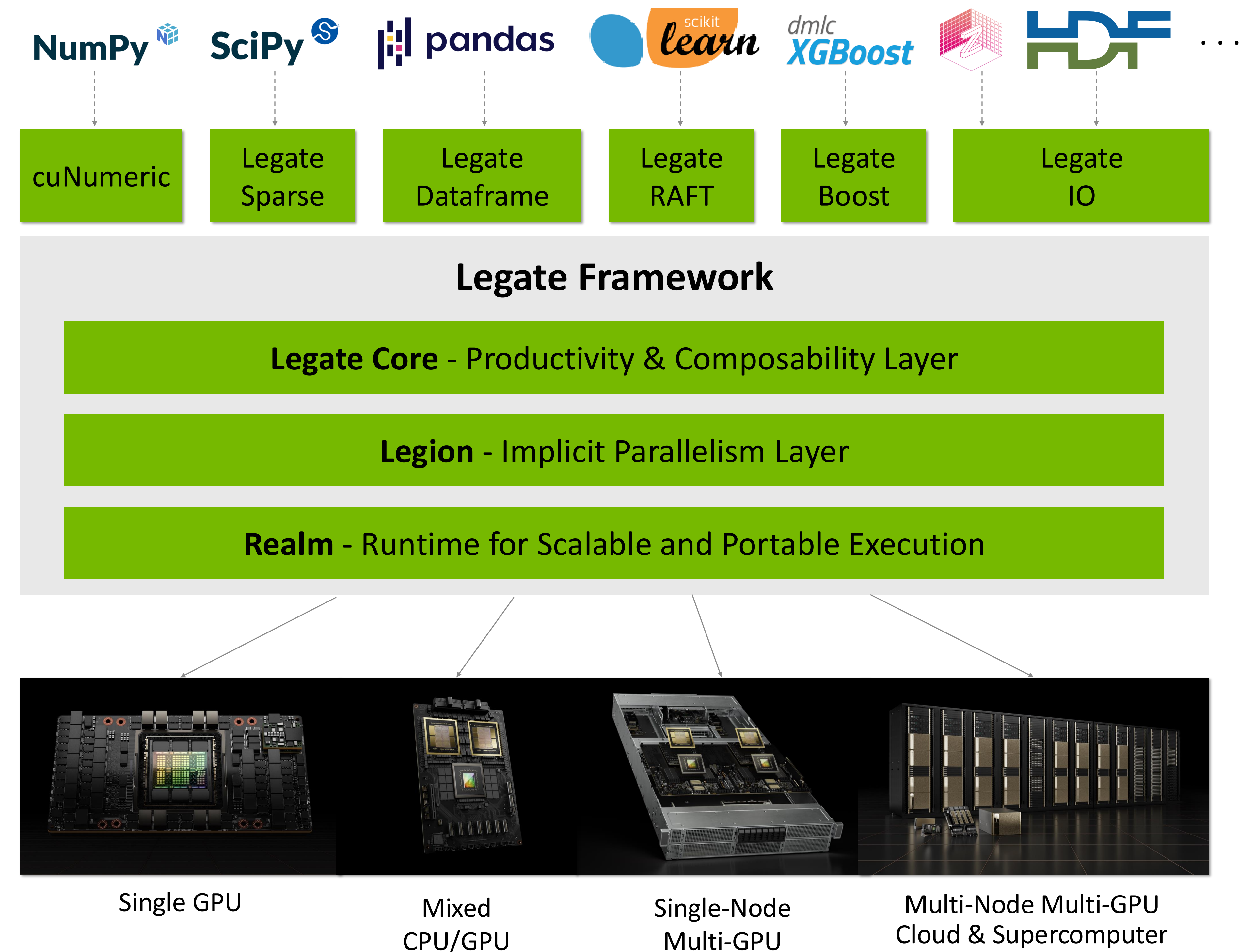
Programming framework for scalable and composable software

Provides uniform solutions to common scaling problems

- Data partitioning and coherence management
- Compute partitioning and distribution
- Scalable execution on distributed memory machines
- **Composability/interoperability between libraries**

Built on years of research on tasking runtimes from the Legion project

- Demonstrated implicit parallelism in the presence of complex data dependence can be made scalable via tasking
- First-class data partitions turn out to be the key ingredient to composability
- Constraint-based partitioning facilitates library-local reasoning for data partitions



How Legate Works

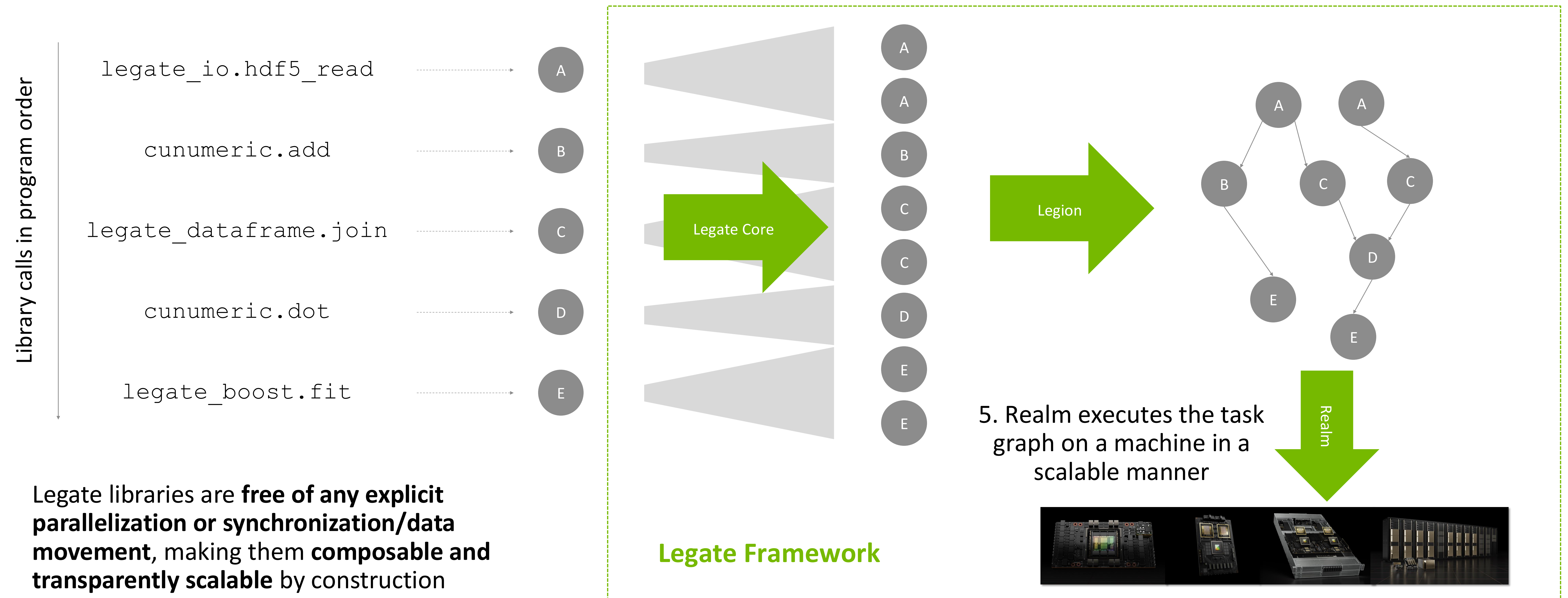
Implicit parallelism via “scale-free” tasking

1. Legate program makes API calls

2. Legate libraries issue “scale-free” tasks

3. Legate core converts each scale-free task to parallel tasks

4. Legion analyzes data dependencies and constructs a Realm task graph



Programming Model

Index launch¹ + constraint-based auto-partitioning² = scale-free tasking

Example: Multi-GPU Batched FFTs

Declare a **task**, a unit of computation applied to a subset of data

Specify desirable data partitions using **partitioning constraints**

Specify how data is used by the task via type annotations

Task bodies can **reuse the existing single-GPU libraries**

Tasks are **invoked like Python functions**, and the runtime **automatically parallelizes** them by **partitioning inputs and outputs**

```
import legate.core as lg

@lg.task(
    variants=("gpu",)
    constraints=(
        lg.align("input", "output"),
        lg.broadcast("input", (0,)),
    ),
)
def batched_fft(
    output: lg.OutputArray, input: lg.InputArray,
):
    cu_input      = cupy.asarray(input)
    cu_output     = cupy.asarray(output)
    cu_output[:] = cupy.fft.fftn(cu_input, axes=(0,))

batched_fft(
    lg.create_array(shape=(M,N), dtype=lg.complex64),
    cunumeric.random.rand(M,N).astype(np.complex64),
)
```

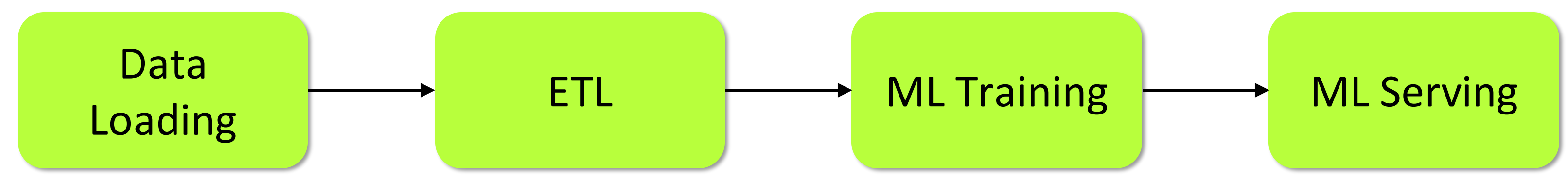
1. Rupanshu Soi, Michael Bauer, Sean Treichler, Manolis Papadakis, Wonchan Lee, Patrick S. McCormick, Alex Aiken, Elliott Slaughter, *Index launches: scalable, flexible representation of parallel task groups*. SC 2021

2. Wonchan Lee, Manolis Papadakis, Elliott Slaughter, Alex Aiken, *A constraint-based approach to automatic data partitioning for distributed memory execution*. SC 2019

Performance evaluation

RAPIDS GPU-XB-AI Benchmark

- Derived from the TPCx-AI benchmark consisting of uses cases with ETL + ML training/serving
- Requires **four different libraries** to **interoperate seamlessly and scale collectively**



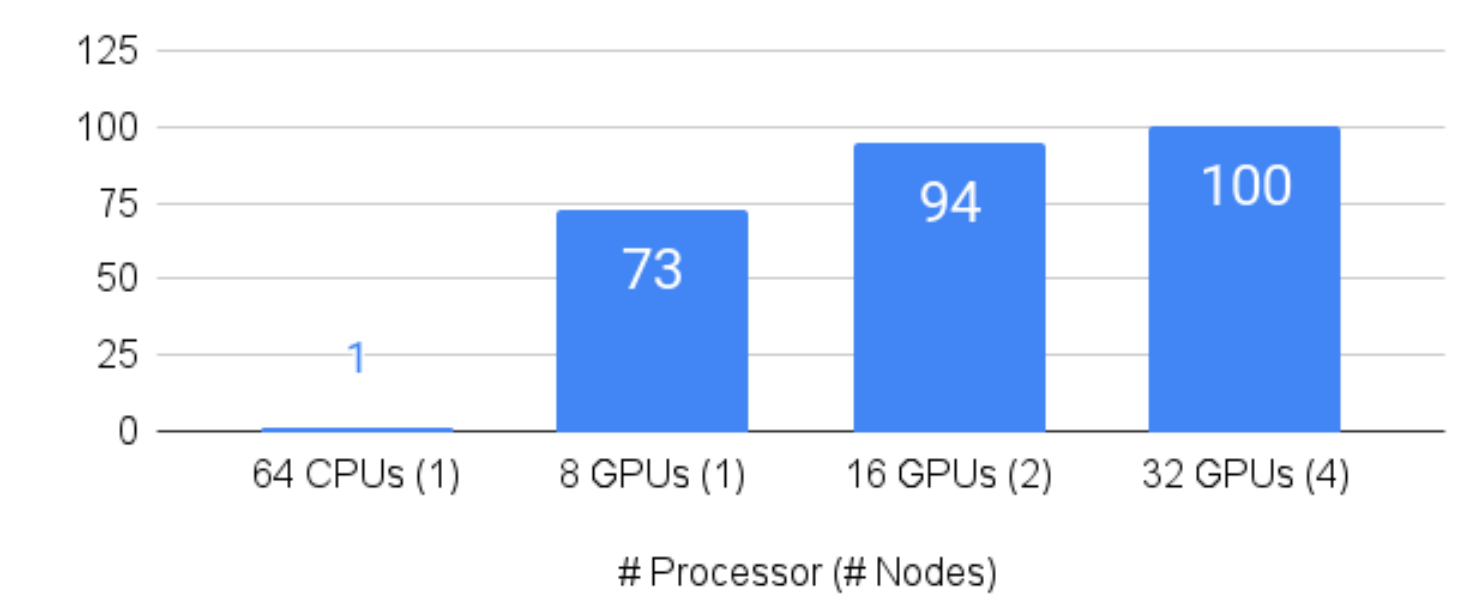
Workload	ETL Legate Dataframe + cuNumeric	ML Legate RAFT + Legate Boost
UC01	Hash join, hash aggregate, null handling, element-wise ops	K-means
UC04	Drop duplicates, cast to string, functions to support TF-IDF	Naive Bayes, TF-IDF
UC10	Hash join, datetime support, element-wise arithmetic	Logistic Regression

Preliminary comparison* with official TPCx-AI results

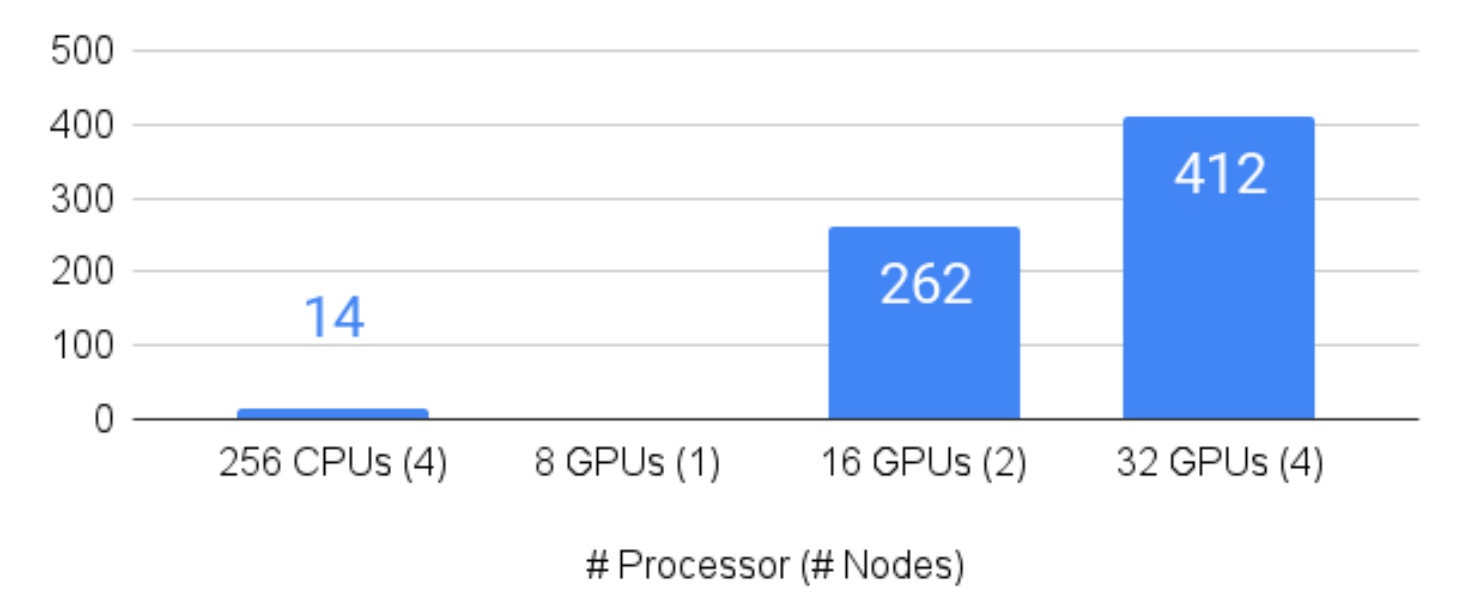
$$Score = \frac{SF \cdot (\# \text{ use cases})}{geomean(Perf_{training}, Perf_{serving})}$$

Cost	CPU nodes	GPU nodes
SF30	\$ 44,248	\$ 270,000 / node
SF300	\$ 309,091	

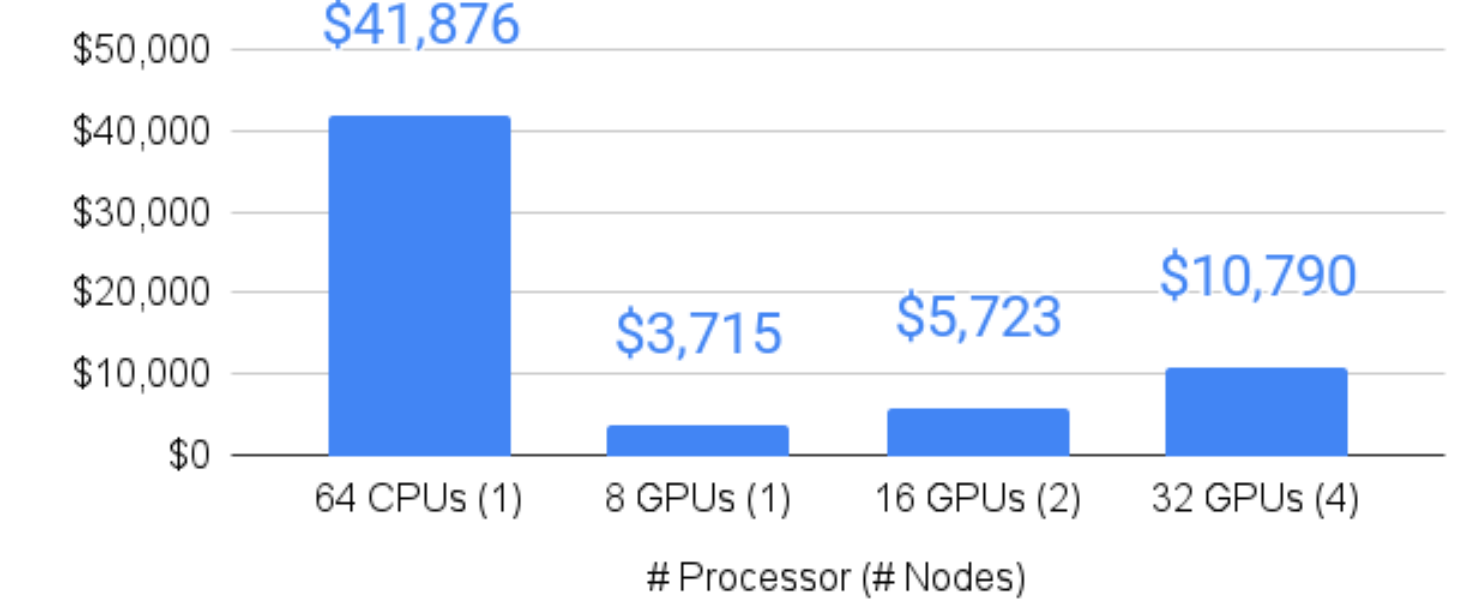
Overall Performance Score - SF30



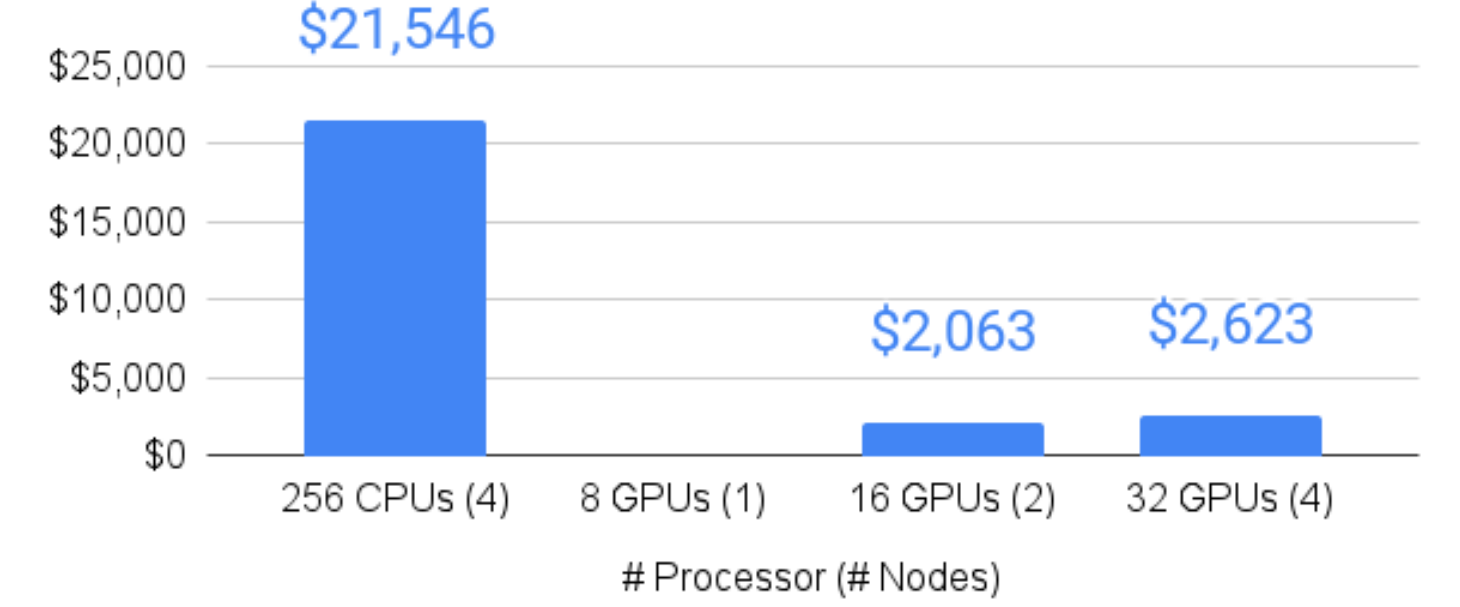
Overall Performance Score - SF300



Cost / Score - SF30



Cost / Score - SF300



Legate can be ~10X more cost efficient than the CPU counterpart

* Disclaimer: the Legate numbers aren't official TPCx-AI results and do not abide by the official rules of TPCx-AI

Conclusion

- Zero code change scaling enables productive programming of distributed memory machines
- Legate framework offers uniform solutions to common scaling problems, thereby making libraries scalable and composable by construction
- Learn more
 - [Legate – Get Started](#)
 - [NVIDIA Legate Core official documentation](#)
 - [Legate presentation at GTC'24](#)
 - [Scaling NumPy from Laptops to Supercomputers Effortlessly with NVIDIA cuPyNumeric](#)
- For any questions, please contact the Legate team on legate@nvidia.com

